

Sistema Esperto per l'Agricoltura di Precisione

Gruppo di lavoro

- Riccardo Paparesta, Matricola:804061,
r.paparesta@studenti.uniba.it

< <https://github.com/KDestinyK/ICON> >

AA 2025-2026

Introduzione

L'idea di questo progetto nasce dall'agricoltura di precisione, un ambito in cui decisioni semplici, se irrigare, quale coltura mettere a dimora, quando intervenire, dipendono in realtà da molti fattori che si intrecciano: l'umidità del suolo, la temperatura, l'umidità dell'aria, ma anche la stagione, la coltura piantata in precedenza e le caratteristiche della specie. Un agricoltore esperto tiene conto di tutto questo quasi automaticamente; l'obiettivo è stato provare a costruire un sistema che ragioni in modo simile.

Il problema affrontato ha due facce. La prima è capire lo stato idrico di un terreno a partire dai dati che un sensore può misurare. La seconda, più interessante per me, è ragionare su quello stato: incrociarlo con le regole agronomiche per arrivare a un consiglio che abbia senso e che si possa spiegare, non a un semplice numero.

Sommario

Il sistema realizzato è un Knowledge-Based System (KBS) costruito attorno a tre moduli che lavorano in sequenza, dove ciò che esce da uno entra nel successivo. Un classificatore predice lo stato idrico del terreno; una rete bayesiana, la cui struttura viene appresa dai dati, ci dice quanto possiamo fidarci di quella predizione viste le condizioni climatiche; infine un programma Prolog mette insieme le due informazioni con la conoscenza agronomica della Knowledge Base e ne ricava l'azione da consigliare.

Il punto su cui ho lavorato di più è proprio il legame tra i moduli. Non si tratta di tre programmi separati messi in fila: la classe predetta dal classificatore e la confidenza calcolata dalla rete bayesiana vengono inserite nella KB come fatti, e sono quei fatti a far scattare le regole di ragionamento. Senza quel passaggio, ciascun modulo da solo non arriverebbe alla conclusione finale.

Elenco argomenti di interesse

- Apprendimento supervisionato, confronto tra più classificatori con cross-validation ripetuta
- Ragionamento e apprendimento con incertezza, rete bayesiana con apprendimento della struttura e inferenza.
- Rappresentazione della conoscenza e ragionamento automatico, ontologia e regole in Prolog basate su clausole di Horn.

Sezione 1: Apprendimento supervisionato

Sommario

In questa sezione il sistema impara a riconoscere lo stato idrico di un terreno a partire dalle sue caratteristiche. I dati vengono dal dataset `cropdata_updated.csv`, che contiene 16.411 rilevazioni su cinque colture (frumento, patata, carota, pomodoro, peperoncino). Ogni riga ha tre attributi descrittivi (coltura, tipo di terreno, stadio di crescita) e tre misure numeriche (umidità del suolo, temperatura, umidità dell'aria), più la colonna `result` che vale 0, 1 o 2.

Il dataset però non spiega cosa significhino quei tre valori: tutta la parte di ragionamento dipende da cosa rappresentano. Per capirlo ho guardato i valori medi di temperatura, umidità e umidità del suolo per ciascuna classe (Tabella 1). La classe 0 raccoglie i casi di terreno umido e clima fresco, dove non serve irrigare; la classe 1 quelli di terreno secco e caldo, dove l'irrigazione serve; la classe 2 i casi di umidità del suolo molto alta insieme ad alta temperatura, cioè una situazione di stress in cui irrigare ancora sarebbe controproducente. È su questa lettura che si basa poi tutta la logica della Sezione 3.

Classe	MOI medio	Temp media	Umidità media	Interpretazione
0	48.3	23.7 °C	74.5	Nessuna irrigazione
1	32.7	35.1 °C	50.1	Irrigazione necessaria
2	67.6	35.4 °C	49.4	Stress idrico

Tabella 1 — Valori medi per classe, da cui ho ricavato il significato del target.

Strumenti utilizzati

Ho confrontato tre classificatori di `scikit-learn` [1]: Random Forest, Support Vector Machine con kernel RBF e Regressione Logistica. Sono modelli noti e ben documentati, quindi non ne riporto la teoria. Il motivo per cui ne ho provati tre, invece di sceglierne uno solo, è che lavorano in modo diverso tra loro: confrontandoli sugli stessi dati potevo decidere quale usare guardando i risultati, anziché a priori.

Decisioni di Progetto

Preprocessing: gli attributi categorici (coltura, terreno, stadio) sono stati trasformati con one-hot encoding, e tutte le feature sono state riportate sulla stessa scala con `StandardScaler`. Questo passaggio serve soprattutto a SVM e regressione logistica, che risentono molto della scala dei valori; per la Random Forest è indifferente, ma l'ho applicato comunque a tutte per uniformità.

Scelta degli iperparametri: invece di fissare a mano i parametri dei modelli, li ho cercati con una `GridSearch` in cross-validation (3 fold, F1 pesato come criterio), che prova tutte le combinazioni di una griglia di valori e tiene la migliore. Il dataset è grande e SVM in particolare diventa lento, così per la ricerca ho usato un campione di circa 4.000 righe; la valutazione vera e propria, invece, è fatta su tutti i dati. I valori che sono risultati migliori sono in Tabella 2.

Modello	Iperparametri esplorati	Valori selezionati
Random Forest	n_estimators, max_depth, criterion	200, illimitata, entropy
SVM	C, kernel, gamma	10, rbf, scale
Logistic Regression	C, solver	10, lbfgs

Tabella 2 — Iperparametri provati e quelli scelti dalla GridSearch.

Come valutare i modelli: un singolo addestramento dice poco, perché il risultato dipende da come capita di dividere i dati. Per questo ho usato la Repeated K-Fold (5 fold ripetuti 5 volte, quindi 25 addestramenti per modello) e riporto media e deviazione standard. Le metriche sono pesate sulle classi, perché la classe 2 è poco rappresentata (il 6,8% dei dati) e una media semplice la nasconderebbe.

Valutazione

Le metriche usate (accuratezza, precision, recall, F1, tutte pesate) sono quelle standard, quindi non le ridefiniamo. I risultati medi sui 25 addestramenti sono questi:

Modello	Accuratezza	Precision	Recall	F1
Random Forest	0.9935 ± 0.0014	0.9935 ± 0.0015	0.9935 ± 0.0014	0.9934 ± 0.0015
SVM	0.9591 ± 0.0028	0.9586 ± 0.0028	0.9591 ± 0.0028	0.9584 ± 0.0029
Logistic Regression	0.8275 ± 0.0066	0.7773 ± 0.0091	0.8275 ± 0.0066	0.8008 ± 0.0076

Tabella 3 — Risultati medi (5-fold × 5 ripetizioni), media ± deviazione standard.

La Random Forest è chiaramente la migliore (F1 = 0.9934) ed è quella che uso nella pipeline. Le deviazioni standard piccolissime ci dicono che il risultato è stabile e non frutto di una divisione fortunata dei dati. Un'accuratezza così alta però è sospetta: poteva trattarsi di overfitting, cioè di un modello che ha imparato a memoria. Per controllare ho guardato la curva di apprendimento, che confronta l'accuratezza sul training e sul test man mano che si aggiungono dati:

Dimensione training	Accuratezza training	Accuratezza test
666	1.0000	0.9174
1333	1.0000	0.9342
1999	1.0000	0.9486
2666	1.0000	0.9616
3333	1.0000	0.9652

Tabella 4 — Curva di apprendimento della Random Forest.

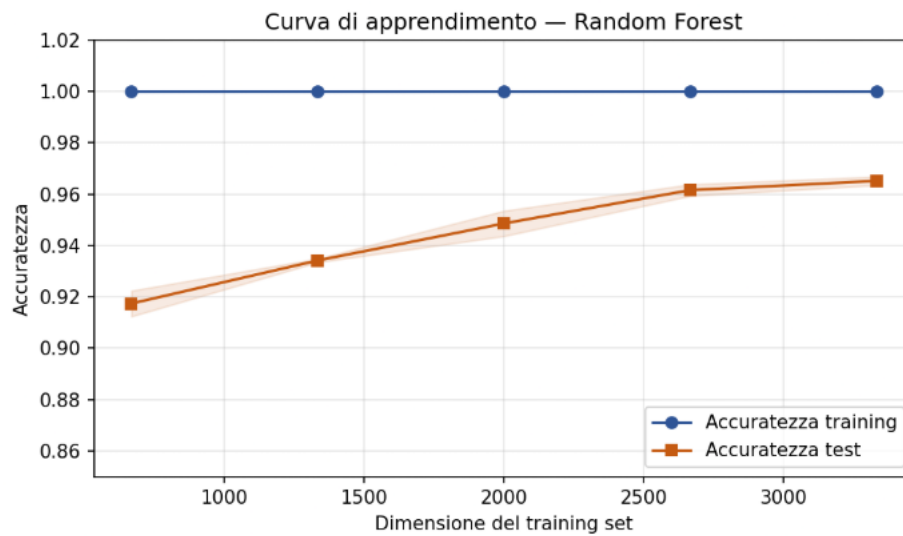


Figura 1 — La curva di test sale e si avvicina a quella di training.

L'accuratezza sul test continua a salire man mano che aggiungiamo dati, e la distanza dalla curva di training si stringe fino a circa 0.035. Se ci fosse stato overfitting il divario sarebbe rimasto ampio; il fatto che si chiuda ci dice che il modello sta davvero generalizzando, non memorizzando. La curva di training resta a 1.0 perché una foresta non potata azzerava sempre l'errore sul training: è normale e non è un problema.

Sezione 2: Ragionamento Probabilistico(rete bayesiana)

Sommario

Qui rappresento come le condizioni ambientali si influenzano a vicenda e influenzano lo stato idrico, usando una rete bayesiana: un grafo in cui ogni nodo è una variabile e ogni freccia una dipendenza, con annessa una tabella di probabilità. Le variabili nascono dalla discretizzazione degli attributi del dataset. La rete serve per due cose: rappresentare in modo esplicito i legami tra clima e stato idrico, e soprattutto dare alla pipeline una misura di quanto fidarsi della predizione del classificatore. Quest'ultima informazione diventa poi il fatto affidabilità_meteo usato da Prolog.

Strumenti utilizzati

Per costruire e interrogare la rete ho usato la libreria pgmpy [2]. Gli algoritmi che entrano in gioco : HillClimbSearch per scoprire la struttura, Maximum Likelihood Estimation per stimare le probabilità e Variable Elimination per l'inferenza , tutti noti quindi mi limito a citarli[4].

Decisioni di Progetto

Discretizzazione: temperatura, umidità dell'aria e umidità del suolo sono valori continui, ma la rete lavora con variabili discrete, quindi ho dovuto trasformarle. Le ho rese binarie (alto/basso) usando la mediana come soglia. Avrei potuto fissare soglie a occhio, ma la mediana ha due vantaggi pratici: divide i campioni a metà, e così nessuno stato resta quasi vuoto, ed evita di trovarci tabelle di probabilità con caselle a zero, che mandano in errore l'inferenza quando incontra un valore mai visto.

Apprendimento della struttura: non ho disegnato io le frecce della rete, le ho fatte trovare a HillClimbSearch a partire dai dati, usando come criterio il punteggio BIC. Il BIC premia le strutture che spiegano bene i dati ma penalizza quelle troppo piene di archi, così si evitano collegamenti spuri. Lo stadio di crescita e il tipo di terreno sono rimasti fuori perché l'algoritmo non li ha ritenuti utili; li ho lasciati fuori volentieri, perché aggiungerli avrebbe gonfiato le tabelle di probabilità rendendo meno affidabili le stime. Alla fine la rete (Figura 2) collega temperatura e umidità allo stato idrico.

Struttura appresa della rete bayesiana (HillClimbSearch + BIC)

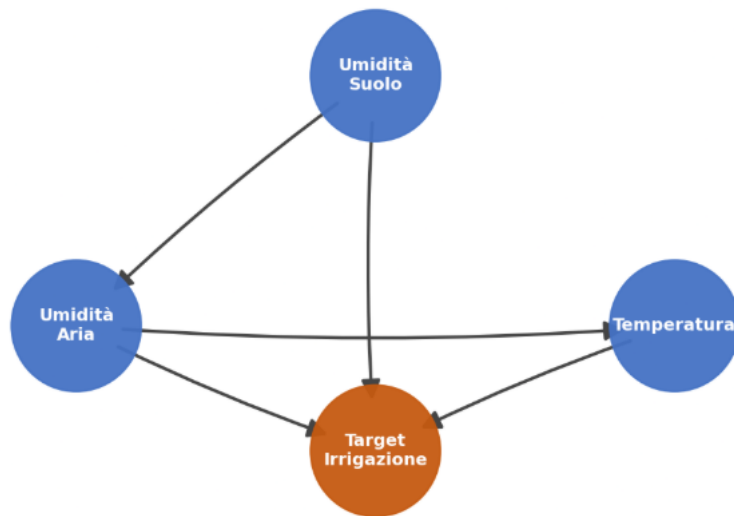


Figura 2 — La struttura trovata da HillClimbSearch. Lo stadio di crescita non compare perché l'algoritmo non l'ha selezionato.

Valutazione

Ho valutato la rete in due modi. Prima di tutto mi sono chiesto se la struttura trovata fosse davvero buona, e l'ho confrontata con una rete Naive Bayes (quella in cui tutto dipende solo dal target) usando il punteggio BIC:

Modello	Punteggio BIC	Esito
Struttura appresa (HillClimb)	≈ -32.947	Migliore
Baseline Naive Bayes	≈ -76.094	Peggior

Tabella 5 — Confronto BIC (più alto è meglio).

La struttura ottiene un BIC molto più alto, segno che le dipendenze trovate da HillClimbSearch descrivono i dati meglio della semplificazione del Naive Bayes. Poi ho provato ad usare la rete come se fosse un classificatore, per vedere quanto è capace di predire da sola, sempre in 5-fold:

Accuratezza	0.756 ± 0.010
F1 pesato	0.718 ± 0.011

Tabella 6 — La rete bayesiana usata come classificatore (5-fold).

Come previsto fa peggio della Random Forest, e va benissimo così. Una rete bayesiana è un modello generativo: cerca di descrivere come sono distribuite tutte le variabili insieme, mentre un modello discriminativo come la foresta punta solo a separare le classi, ed è naturale che in classificazione vinca. Ma la rete non serve per classificare, di quello si occupa già il modulo precedente. Serve per un'altra cosa: dire quanto è probabile, viste le condizioni climatiche, che la predizione del classificatore sia giusta, e per gestire i casi in cui non abbiamo tutti i dati. È questa informazione che passa al ragionamento logico.

Sezione 3: Rappresentazione e ragionamento logico(KB)

Sommario

La Knowledge Base in Prolog è il punto in cui tutto si tiene insieme: qui i risultati numerici dei due moduli precedenti incontrano la conoscenza agronomica e diventano un consiglio. Ho scelto una rappresentazione a ontologia: le 16 colture sono raggruppate in 7 famiglie botaniche tramite il predicato `cultura/2` (per esempio `cultura(pomodoro, solanacee)`). Non l'ho fatto per ordine, ma perché conviene al ragionamento: molte proprietà, tolleranza alla siccità, concimazione, regole di rotazione, valgono per l'intera famiglia, non per la singola pianta. Così, per aggiungere una nuova coltura, basta un fatto: erediterà da sola tutte le regole della sua famiglia.

Strumenti utilizzati

Il motore logico è SWI-Prolog, che uso da Python tramite `pyswip` [3]. Prolog e la risoluzione su clausole di Horn sono noti [5], quindi non mi dilungo. Vale la pena sottolineare una cosa: non ho inventato nessun motore di inferenza, quello è quello standard di Prolog; la parte originale è la conoscenza agronomica e il modo in cui le regole combinano i dati dei due moduli.

Decisioni di Progetto

Fatti dinamici: accanto ai fatti fissi (la gerarchia delle famiglie, i mesi di semina, le tolleranze, i fabbisogni, i terreni adatti), la KB riceve a ogni esecuzione due fatti che arrivano dalla pipeline: `bisogno_acqua/1` (sì o no, da cosa ha predetto il classificatore) e `affidabilita_meteo/1` (alta o bassa, dalla probabilità della rete bayesiana). Sono questi due fatti, che cambiano da un caso all'altro, a far ragionare le regole fisse.

Le regole: la regola principale è `azione_operativa/4`, che per arrivare al consiglio mette insieme tre cose — se la coltura si può seminare (cosa che a sua volta dipende dalla stagione e dalla rotazione), cosa ha detto il classificatore sull'acqua, e quanto è affidabile secondo la rete bayesiana. Per fare un esempio concreto: la regola dell'"irrigazione di soccorso" scatta solo quando il classificatore chiede acqua, la rete però segnala bassa affidabilità e la coltura appartiene a una famiglia che soffre la siccità. Tre informazioni che vengono da tre moduli diversi e che si incontrano in un'unica conclusione. Un altro esempio è la regola di rotazione: due colture sono incompatibili se appartengono alla stessa famiglia. Non ho scritto da nessuna parte "pomodoro è incompatibile con patata"; il sistema lo deduce da solo perché entrambe sono solanacee.

Un accorgimento tecnico: le regole che usano la negazione (`"\+"`, cioè "non è dimostrabile che...") possono dare problemi se le variabili non sono ancora legate a un valore. Le ho quindi protette con `nonvar/1`, che controlla l'istanziamento prima di applicare la negazione, evitando errori durante la ricerca della soluzione.

Valutazione

Il ragionamento logico non è un task statistico. Quello che conta è la profondità del ragionamento, cioè quante deduzioni si concatenano per arrivare a una conclusione, e non quanti fatti ci sono nella KB. Per dare un'idea: valutare `azione_operativa/4` richiama `puo_seminare/3`, che a sua volta interroga `meso_semina/2` e `nega_incompatibile/2`, che infine risale la gerarchia con due passaggi su `cultura/2`. Non è quindi un semplice confronto su fatti già scritti, ma una catena di passi.

Per verificare che funzionasse, ho provato il sistema su tre scenari, uno per ciascuna classe, presi dal dataset. A questi ho aggiunto a mano il mese e la coltura precedente, due informazioni che nel dataset non ci sono: servivano per mettere alla prova le regole di stagione e rotazione, è una scelta, non un dato reale. In tutti e tre i casi il classificatore ha indovinato la classe e Prolog ha risposto in modo sensato: ha segnalato il conflitto di rotazione tra pomodoro e patata (sono entrambe solanacee), ha riconosciuto lo stress idrico e ha consigliato di non irrigare nonostante il caldo. Infine, l'advisor di rotazione — che suggerisce cosa seminare in un certo mese su un certo terreno usando solo la KB — mostra che la base di conoscenza ha senso anche da sola, fuori dalla pipeline.

Conclusioni

Alla fine il sistema fa quello che deve: tiene insieme i tre moduli facendoli parlare tra loro, con i dati che passano dall'apprendimento alla rete bayesiana fino al ragionamento logico. Ogni pezzo è stato valutato con gli strumenti adatti, cross-validation ripetuta con media e deviazione standard per i classificatori, confronto BIC e prova predittiva per la rete, analisi delle catene di inferenza per la KB. Il classificatore va molto bene e in modo stabile, la rete bayesiana impara una struttura nettamente migliore della baseline, e la KB ragiona in modo non banale appoggiandosi alla gerarchia delle famiglie.

Restano alcune cose che sono possibili estensioni:

- ☐ collegare la KB a un'ontologia esterna (OWL) e ricavare i fatti agronomici tramite ragionamento automatico, invece di scriverli a mano;
- ☐ trovare un dataset con informazioni di tempo reali (data di semina, storico delle coltivazioni), così da non dover più simulare il contesto negli scenari;
- ☐ aggiungere un'interfaccia con cui un agronomo possa interrogare il sistema sul proprio campo e segnalare com'è andata, in modo da migliorarlo nel tempo.

Riferimenti Bibliografici

[1] Documentazione di scikit-learn — <https://scikit-learn.org>

[2] Documentazione di pgmpy — <https://pgmpy.org>

[3] Documentazione di pyswip e SWI-Prolog — <https://www.swi-prolog.org>

[4] D. Poole, A. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3 ed., Cambridge University Press [capp. 7, 9, 10].

[5] D. Poole, A. Mackworth, Artificial Intelligence: Foundations of Computational Agents, 3 ed., Cambridge University Press [capp. 5, 15].